

Тема «Разработка интерфейсов на Python»

Запуск оконного интерфейса

В языке Python модуль, который вы можете использовать для создания графических приложений, называется **tkinter** (**tool**kit to **inter**face, набор инструментов для интерфейса).

Чтобы обеспечить атрибуты и методы для создания оконного интерфейса, **tkinter**, как и другие модули, должен быть импортирован в программу на Python. Каждая программа, использующая ресурсы данного модуля, должна начинаться с вызова конструктора **Tk()**, создающего объект окна. При помощи метода **geometry()** этого объекта можно дополнительно указать размеры окна, передав в качестве аргумента строку **'ширинахвысота'**. Можно также создать заголовок для окна, указав методу **title()** строковый аргумент **'заголовок'**. Если размеры и заголовок не указаны, то будут использоваться значения, заданные по умолчанию.

Завершать каждую программу должен так называемый цикл обработки событий окна, вызываемый методом **mainloop()**, который реагирует на действия пользователя с окном в течение исполнения программы (закрытие окна для выхода из программы или просто изменение его размеров).

Все элементы управления графического интерфейса, создаваемые с использованием модуля **tkinter**, такие как кнопки или флажки, называют **виджетами**. Самым простым примером виджета является **Label**, который отображает обычный текст или изображение в интерфейсе приложения. Создается он при помощи конструктора **Label()**, которому в качестве аргументов указывается имя окна и текст для самой метки в виде **text='строка'**.

После создания любого виджета его нужно поместить на окно. Для этого в Python существуют специальные методы, называемые **менеджерами размещений**.

- **pack()** — располагает виджеты по указанной стороне окна при помощи параметра **side=**, который может принимать значения четырех констант: **TOP**, **BOTTOM**, **LEFT** и **RIGHT**.
- **place()** — помещает виджет в точку окна с координатами X,Y, указанными ему в виде параметров **x=**, **y=**, заданными числовыми значениями в пикселах.
- **grid()** — позволяет разместить виджет в ячейку таблицы в соответствии с указанными числовыми параметрами **row=**, **column=**, определяющими номер строки и номер столбца этой ячейки.

Метод `pack()` может принимать также дополнительные аргументы, включая `fill`, определяющий возможность виджета заполнять свободное пространство вдоль какой-либо из осей, например вдоль оси x: `fill='x'`, а также аргументы `padx`, `pady`, задающие расширение виджета вдоль соответствующих осей на определенное количество пикселей.

Пример

```
from tkinter import *  
  
window = Tk()  
  
window.title( 'Label Example' )  
  
label = Label( window , text = 'Hello World!' )  
  
label.pack( padx = 400 , pady = 100 )  
  
window.mainloop()
```

Работа с кнопками

Виджет **Button** — элемент графического оконного интерфейса «кнопка», который может содержать либо текст, либо изображение, передающие смысловое назначение кнопки. Объект **Button** создается с помощью указания конструктору `Button()` имени окна и опций в качестве аргументов. Каждая опция определяется как пара *опция=значение*. Определяющей является опция `command`, в которой указывается имя функции или метода, которые должны быть вызваны, когда пользователь нажимает на кнопку. Наиболее популярные опции и их краткое описание приведены в таблице ниже.

Опция	Описание
<code>activebackground</code>	Цвет фона активного элемента
<code>activeforeground</code>	Цвет переднего плана активного элемента
<code>bd</code>	Ширина рамки в пикселах (значение по умолчанию: 2)
<code>bg</code>	Цвет фона
<code>command</code>	Функция, вызываемая при нажатии
<code>fg</code>	Цвет переднего плана
<code>font</code>	Шрифт для метки кнопки
<code>height</code>	Высота кнопки (для текста в количестве строк, для изображений — в пикселах)
<code>highlightcolor</code>	Цвет рамки при наведении
<code>image</code>	Изображение для вывода вместо текста
<code>justify</code>	Вид выравнивания (по левому краю, по центру, по правому краю)
<code>padx</code>	Количество пикселей до края по горизонтали
<code>pady</code>	Количество пикселей до края по вертикали
<code>relief</code>	Вид рельефности рамки (SUNKEN — утопленная, RIDGE — выпуклая кайма, RAISED — выпуклая, GROOVE — канавка)
<code>state</code>	Состояние (NORMAL — рабочее или DISABLED — отключена)
<code>underline</code>	Порядковый номер символа в тексте, который необходимо подчеркнуть (значение по умолчанию: 1)
<code>width</code>	Ширина кнопки (в символах для текста, в пикселах для изображения)
<code>wraplength</code>	Параметр, определяющий ширину, в которую вписывается текст

Значения, присвоенные различным опциям, определяют внешний вид виджета. Опции разрешается изменять с помощью метода виджета `configure()`, указав ему в качестве аргумента новое значение пары **опция=значение**. Кроме того, можно получить значение каждой опции с помощью метода виджета `cget()`, указав в качестве строкового аргумента имя определенной опции.

Пример

```
from tkinter import *
window = Tk()
window.title( 'Button Example' )
btn_end = Button( window , text = 'Close' , command=exit )
def tog():
    if window.cget( 'bg' ) == 'yellow' :
        window.configure(bg = 'gray')
    else :
        window.configure(bg = 'yellow')
btn_tog = Button( window , text = 'Switch' , command=tog )
btn_end.pack( padx = 150 , pady = 20 )
```

```
btn_tog.pack( padx = 150 , pady = 20 )
```

```
window.mainloop()
```

Вывод сообщений

В программе на Python можно выводить сообщения пользователю в виде диалоговых окон с помощью методов, представляемых модулем `tkinter.messagebox`. Данный модуль должен быть импортирован отдельно, а вместо его длинного имени допускается назначить псевдоним с помощью инструкции `import as`.

Диалоговое окно может быть создано с помощью указания заголовка окна, а также самого сообщения в виде двух аргументов для одного из следующих методов.

Метод	Значок	Кнопки
<code>showinfo()</code>		ОК
<code>showwarning()</code>		ОК
<code>showerror()</code>		ОК
<code>askquestion()</code>		Yes (возвращает строку 'yes') и No (возвращает строку 'no')
<code>askokcancel()</code>		ОК (возвращает 1) и Cancel
<code>askyesno()</code>		Yes (возвращает 1) и No
<code>askretrycancel()</code>		Retry (возвращает 1) и Cancel

Методы, которые выводят диалоговое окно, содержащее единственную кнопку ОК, не возвращают никакого значения при ее нажатии пользователем. В тех методах, где возвращается значение, можно использовать данное возвращаемое значение для дальнейшего условного ветвления.

Пример

```
from tkinter import *  
import tkinter.messagebox as box  
window = Tk()  
window.title( 'Button Example' )  
def dialog() :
```

```

var = box.askyesno( 'Message Box' , 'Proceed?' )
if var == 1 :
    box.showinfo( 'Yes Box', 'Proceeding...' )
else :
    box.showwarning( 'No Box', 'Cancelling...' )
btn = Button( window , text = 'Click' , command=dialog )
btn.pack( padx = 150 , pady = 50 )
window.mainloop()

```

Прием данных от пользователя

Для получения данных, введенным пользователем в приложении с графическим интерфейсом, в модуле **tkinter** существует виджет **Entry**, который предоставляет однострочное поле ввода. Объект ввода создается при помощи конструктора **Entry()** указанием ему в качестве аргументов имени родительского контейнера (например, окна или фрейма), а также используемых опций, каждая из которых передается в виде пары **опция=значение**. Наиболее применяемые опции и их краткое описание представлены в таблице ниже:

Опция	Описание
bd	Ширина рамки в пикселах (значение по умолчанию: 2)
bg	Цвет фона
fg	Цвет переднего плана
font	Шрифт для текста
highlightcolor	Цвет рамки при наведении
selectbackground	Цвет фона выделенного текста
selectforeground	Цвет переднего плана выделенного текста
show	Использовать вместо видимых символов маскирующие
state	Состояние (NORMAL — рабочее или DISABLED — отключена)
width	Ширина поля ввода в символах

В целях оптимизации размещения несколько виджетов можно сгруппировать во фреймы. Объект фрейм создается при помощи конструктора **Frame()** указанием ему в качестве аргумента имени окна. После этого имя фрейма может быть передано первым аргументом конструктору виджета, чтобы указать, что данный фрейм является для виджета контейнером. При добавлении виджета на фрейм вы можете указывать его привязку к определенной стороне фрейма, используя константы **TOP**, **BOTTOM**, **LEFT** и **RIGHT**. Например, **entry.pack(side=LEFT)**.

Как правило, виджет Entry для ввода текста размещают рядом с меткой, в которой описывается, что должен вводить пользователь, или рядом с кнопкой, которую пользователь может нажать, чтобы выполнить какие-то действия над введенными им данными. Поэтому размещение виджетов в одном фрейме является оптимальным вариантом.

Данные, введенные пользователем в виджете **Entry**, можно получить в программе, используя метод виджета **get()**.

Пример

```
from tkinter import *
import tkinter.messagebox as box

window = Tk()
window.title( 'Entry Example' )
frame = Frame( window )
entry = Entry( frame )
def dialog() :
    box.showinfo( 'Greetings' , 'Welcome, ' + entry.get() )
btn = Button( frame, text = 'Enter Name' , command=dialog )
btn.pack( side = RIGHT , padx = 5 )
entry.pack( side = LEFT )
frame.pack( padx = 20 , pady = 20 )
window.mainloop()
```

Организация выбора из списка

При помощи виджета **Listbox** вы можете добавлять в приложение список элементов, предлагаемых пользователю для выбора. Для создания объекта **listbox** используется конструктор **Listbox()**, которому в качестве аргументов указываются имя родительского контейнера (например, окна или фрейма) и возможные опции, самые популярные из которых представлены вместе с кратким описанием в таблице ниже.

Опция	Описание
<code>bd</code>	Ширина рамки в пикселах (значение по умолчанию: 2)
<code>bg</code>	Цвет фона
<code>fg</code>	Цвет переднего плана
<code>font</code>	Шрифт для текста
<code>height</code>	Количество строк в списке (значение по умолчанию: 10)
<code>selectbackground</code>	Цвет фона выделенного текста
<code>selectmode</code>	Режим выбора (SINGLE — одиночный по умолчанию или MULTIPLE — множественный)
<code>width</code>	Ширина поля ввода списка в символах (значение по умолчанию: 20)
<code>yscrollcommand</code>	Привязка к полосе прокрутки

В модуле **tkinter** полоса прокрутки организуется при помощи отдельного виджета **scrollbar**, который необходимо привязывать к виджетам **Listbox**, **Text**, **Canvas** и **Entry**.

Элементы добавляются в список при помощи метода **Insert()**, которому в качестве аргументов указывается порядковый номер элемента в списке и строка, определяющая сам элемент. Любой элемент списка можно получить с помощью метода **get()**, указав его порядковый номер в качестве аргумента. Кроме того, у объекта **listbox** существует полезный метод **curselection()**, возвращающий порядковый номер текущего выбранного элемента, так что его можно использовать как аргумент для метода **get()**, чтобы получить текущий выбор.

Пример

```
from tkinter import *
import tkinter.messagebox as box

window = Tk()
window.title( 'Listbox Example' )
frame = Frame( window )
listbox = Listbox( frame )
listbox.insert( 1 , 'HTML5 in easy steps' )
listbox.insert( 2 , 'CSS3 in easy steps' )
listbox.insert( 3 , 'JavaScript in easy steps' )
def dialog() :
    box.showinfo( 'Selection' , 'Your Choice: ' + \
        listbox.get( listbox.curselection() ) )
```

```
btn = Button( frame, text = 'Choose' , command = dialog )  
btn.pack( side = RIGHT , padx = 5 )  
listbox.pack( side = LEFT )  
frame.pack( padx = 30 , pady = 30 )  
window.mainloop()
```

Использование переключателей

Виджет **Radiobutton** позволяет добавить в графическое приложение один элемент положения переключателя, в которое тот может быть установлен пользователем. Если используется группа таких положений, то пользователь имеет возможность выбрать из этой группы только одно. Объекты положений переключателя группируются вместе, когда необходимо задать одну управляющую переменную и присвоить ей значение в зависимости от выбора пользователя. Для создания такой переменной-объекта применяется конструктор **StringVar()**, создающий пустую строку, или конструктор **IntVar()**, инициализирующий пустую целочисленную переменную-объект.

Для хранения значений,
переданных от выбранного
положения переключателя,
используются не обычные
переменные, а специальные
объекты, выполняющие
роль переменных.

Объект положения переключателя создается при помощи конструктора **Radiobutton()**, которому передаются четыре аргумента:

- имя родительского контейнера, например фрейма;
- текстовая строка, которая будет являться меткой, передаваемая в виде пары **text='текст'**;
- управляющая переменная-объект, указываемая в виде пары **variable=имя-переменной**;
- значение для присваивания, передаваемое в виде пары **value=значение**.

Каждый объект положения переключателя содержит метод **select()**, который может быть использован, чтобы определить, какая кнопка в группе выбрана по умолчанию при запуске программы. Строковое значение, которое присвоилось в результате выбора переключателя, может быть получено из переменной-объекта методом **get()**.

Пример

```
from tkinter import *
import tkinter.messagebox as box

window = Tk()
window.title( 'Radio Button Example' )
frame = Frame( window )
book = StringVar()

radio_1 = Radiobutton( frame , text = 'HTML5' , variable = book , value =
'HTML5 in easy steps' )
radio_2 = Radiobutton( frame , text = 'CSS3' ,variable = book , value =
'CSS3 in easy steps' )
radio_3 = Radiobutton( frame , text = 'JS' ,variable = book , value =
'JavaScript in easy steps' )
radio_1.select()

def dialog() :
    box.showinfo( 'Selection' , 'Your Choice: \n' + book.get() )

btn = Button( frame, text = 'Choose' , command = dialog )
btn.pack( side = RIGHT , padx = 5 )
radio_1.pack( side = LEFT )
radio_2.pack( side = LEFT )
radio_3.pack( side = LEFT )
frame.pack( padx = 30 , pady = 30 )
window.mainloop()
```

Флажки

Виджет **Checkbutton** обеспечивает добавление в графическое приложение одного элемента «флажок», который может быть установлен (выбран) пользователем. Обычно используется группа таких флажков, и пользователь имеет возможность выбрать из этой группы один или несколько. Объекты флажков задают каждый свою управляющую переменную-объект, и ей присваивается значение в соответствии с тем, отмечен флажок пользователем или нет. Для создания такой переменной-объекта применяется конструктор **StringVar()**, создающий пустую строку, или конструктор **IntVar()**, инициализирующий пустую целочисленную переменную-объект.

Объект флажка создается при помощи конструктора **Checkbutton()**, которому передаются пять аргументов.

- Имя родительского контейнера, например фрейма.
- Текстовая строка, которая будет являться меткой, передаваемая в виде пары **text='text'**.
- Управляющая переменная-объект, указываемая в виде пары **variable=имя-переменной**.
- Значение для присваивания, в случае если флажок установлен пользователем, передаваемое в виде пары **onvalue=значение**.
- Значение для присваивания, в случае если флажок пользователем не установлен (сброшен), передаваемое в виде пары **offvalue=значение**.

Целочисленное значение, которое присвоено в результате установки флажка, может быть получено из переменной-объекта методом **get()**.

Пример

```
from tkinter import *
import tkinter.messagebox as box
window = Tk()
window.title( 'Check Button Example' )
frame = Frame( window )
var_1 = IntVar()
var_2 = IntVar()
var_3 = IntVar()
book_1 = Checkbutton( frame , text = 'HTML5' , variable = var_1 , onvalue =
1, offvalue = 0 )
book_2 = Checkbutton( frame , text = 'CSS3' , variable = var_2 , onvalue =
1, offvalue = 0 )
```

```
book_3 = Checkbutton( frame , text = 'JS' , variable = var_3 , onvalue = 1,
offvalue = 0 )

def dialog():
    str = 'Your Choice:'
    if var_1.get() == 1 : str += '\nHTML5 in easy steps'
    if var_2.get() == 1 : str += '\nCSS3 in easy steps'
    if var_3.get() == 1 : str += '\nJavaScript in easy steps'
    box.showinfo( 'Selection' , str )

btn = Button( frame, text = 'Choose' , command = dialog )
btn.pack( side = RIGHT , padx = 5 )
book_1.pack( side = LEFT )
book_2.pack( side = LEFT )
book_3.pack( side = LEFT )
frame.pack( padx = 30 , pady = 30 )
window.mainloop()
```

Внедрение графических объектов

Модуль `tkinter` позволяет работать с файлами изображений в форматах GIF или PGM/PPM, которые могут быть выведены на виджетах `Label`, `Text`, `Button` или `Canvas`. Для этих целей используется конструктор `PhotoImage()`, который создает объект изображения. Достаточно указать ему в качестве аргумента имя нужного файла с изображением в виде `file = 'имя-файла'`. Можно воспользоваться полезным методом `subsample()` для уменьшения изображения, указав в качестве аргументов параметры дискретизации по горизонтали и вертикали в виде `x=значение` и `y=значение`. Например, указанные значения `x=2`, `y=2` приведут к отбрасыванию каждого второго пиксела, то есть изображение уменьшится наполовину по отношению к оригиналу.

После того как объект изображения создан, его можно добавлять на виджеты `Label` или `Button` при помощи указания опции `image=` в соответствующих конструкторах.

У объектов виджетов `Text` существует метод `image_create()`, при помощи которого изображение встраивается в текстовое поле. Данный метод принимает два аргумента: первый — для определения позиции размещения (например, `'1.0'` указывает первую строку и первый символ), второй — непосредственная ссылка на само изображение в виде опции `image=`.

Объекты `Canvas` имеют аналогичный метод `create_image()`, принимающий тоже два аргумента, только первый из них, отвечающий за расположение, представлен в виде пары координат `(x,y)`, которые определяют точку на холсте (элементе `Canvas`), куда помещается изображение.

Пример

```
from tkinter import *
window = Tk()
window.title( 'Image Example' )
img = PhotoImage(file = '1.gif')
label = Label( window , image = img , bg = 'yellow' )
small_img = PhotoImage.subsample(img , x = 5 , y = 5 )
btn = Button( window , image = small_img )
txt = Text( window , width = 25 , height = 7 )
txt.image_create( '1.0' , image = small_img )
txt.insert( '1.1', 'Python Fun!' )
can = \
```

```
Canvas( window , width = 100 , height = 100 , bg = 'cyan' )
can.create_image( ( 50 , 50 ), image = small_img )
can.create_line( 0 , 0 , 100 , 100, width = 10 , fill = 'green' )
label.pack( side = TOP )
btn.pack( side = LEFT , padx = 10 )
txt.pack( side = LEFT )
can.pack( side = LEFT, padx = 10 )
window.mainloop()
```